SVD and Representation Learning

ES 335 / ES 678 Machine Learning, Sem I 2026 Sources: the syllabus calendar lists "PCA and representation learning: derive PCA intuition; use eigenvectors/SVD for dimensionality reduction and denoising". The course handbook `sources/course-2026/lectures/L13b_unsupervised_learning_handbook.pdf` covers the same SVD material in section 4.2.5. The rest of these notes are written from standard material and build on the eigen-direction notes.

What this file is really about:

$$\text{any matrix} \;\to\; \text{rotate, stretch, rotate} \;\to\; \text{keep the big stretches} \;\to\; \text{compression, denoising, embed}$$

Step-up path

You need. Eigenvalues, PCA and matrix multiplication.

1. Extend eigen-thinking to rectangular matrices, using two sets of directions.
2. See any matrix as rotate, stretch, rotate.
3. Keep only the biggest stretches, which gives the best low-rank approximation.
4. Identify PCA as the SVD of the centred data.
5. Use the same coordinates as a learned representation.

You can stop when you can compute a small SVD by hand and say how many layers to keep, and why.

Contents

1. Why eigenvectors are not enough
2. The singular value decomposition
3. Computing an SVD by hand
4. Low-rank approximation
5. PCA is an SVD of the centred data
6. Denoising
7. The pseudoinverse and ridge through the SVD
8. Representation learning and embeddings
9. Limits of linear representations

A1 Why eigenvectors are not enough

An eigenvector needs a **square** matrix, because Av must live in the same space as v. A data matrix $X \in \mathbb{R}^{n \times d}$ is usually not square: its rows live in $\mathbb{R}^d$ and its columns in $\mathbb{R}^n$. It maps one space to a different one.

Square or not, every matrix has a simple description, once you allow **two different sets of directions**: one in the input space and one in the output space.

A2 The singular value decomposition

Every real matrix $A \in \mathbb{R}^{m \times n}$ can be written

$$A = U \Sigma V^\top,$$

where U has orthonormal columns u_i (output directions), V has orthonormal columns v_i (input directions), and Σ is diagonal with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, the **singular values**.

Column by column, this says

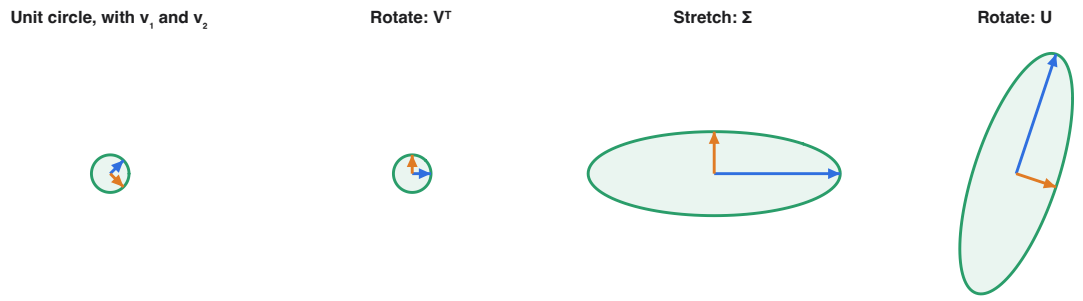
$$Av_i = \sigma_i u_i.$$

Read it as a recipe: **rotate** with $V^\top$, **stretch** the axes with Σ, **rotate** again with U. A unit circle becomes an ellipse, whose semi-axes have lengths σ_i and point along u_i.

The **rank** of A is the number of nonzero singular values.

Written as a sum, the matrix is a stack of rank-one layers, biggest first:

$$A = \sum_i \sigma_i \, u_i v_i^\top.$$



The SVD of the worked matrix in four frames. The blue and orange arrows are the input directions v_1 and v_2. They become axis-aligned, are stretched to lengths 6.71 and 2.24, and finally point along the semi-axes of the ellipse A produces.

A3 Computing an SVD by hand

Multiply on the left by the transpose:

$$A^\top A = V\Sigma^\top U^\top U\Sigma V^\top = V\Sigma^2 V^\top.$$

So the v_i are eigenvectors of $A^\top A$, and σ_i^2 are its eigenvalues. Then $u_i = Av_i/\sigma_i$.

Worked example. $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$.

$A^\top A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ has eigenvalues 45 and 5, with eigenvectors $v_1 = \frac{1}{\sqrt{2}}(1,1)$ and $v_2 = \frac{1}{\sqrt{2}}(1,-1)$.

So $\sigma_1 = \sqrt{45} \approx 6.708$ and $\sigma_2 = \sqrt{5} \approx 2.236$.

Then $u_1 = Av_1/\sigma_1 = \frac{1}{\sqrt{10}}(1,3)$ and $u_2 = Av_2/\sigma_2 = \frac{1}{\sqrt{10}}(3,-1)$.

Check. The squared Frobenius norm, the sum of all squared entries, equals the sum of squared singular values: $9 + 0 + 16 + 25 = 50 = 45 + 5$.

A4 Low-rank approximation

Keep only the first k layers:

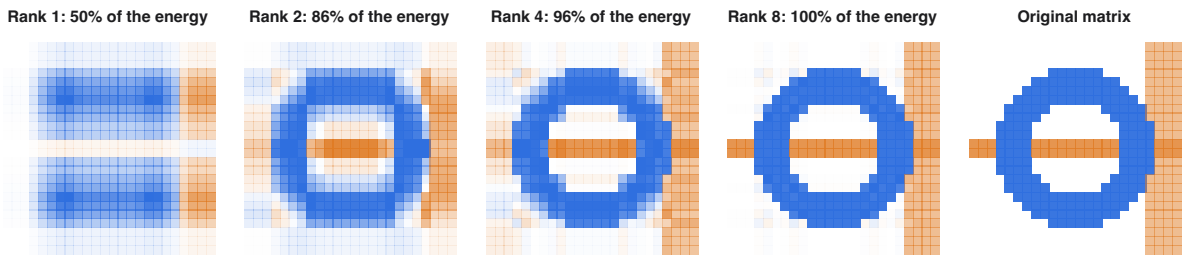
$$A_k = \sum_{i \le k} \sigma_i u_i v_i^\top.$$

Eckart-Young theorem. Among all matrices of rank at most k, A_k is the closest to A in Frobenius norm, and

$$\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2.$$

Worked example, continued. $A_1 = \sigma_1 u_1 v_1^\top = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$. The error is $\begin{bmatrix} 1.5 & -1.5 \\ -0.5 & 0.5 \end{bmatrix}$, whose squared norm is $2.25 + 2.25 + 0.25 + 0.25 = 5 = \sigma_2^2$.

Storage falls from mn numbers to about $k(m+n)$.



Each panel adds more rank-one layers to the same matrix. Blue cells are positive and orange cells negative. Most of the structure arrives in the first few layers, because singular values fall quickly.

A5 PCA is an SVD of the centred data

Let X be centred, with $X = U\Sigma V^\top$. The covariance matrix is

$$S = \tfrac{1}{n} X^\top X = V \, \tfrac{\Sigma^2}{n} \, V^\top.$$

So:

- the principal directions are the columns of V;
- the variance along direction i is $\lambda_i = \sigma_i^2 / n$;
- the coordinates (scores) of the data are $XV = U\Sigma$.

In practice PCA is computed by SVD, not by forming $X^\top X$, because forming that product **squares the condition number** and loses accuracy in small directions.

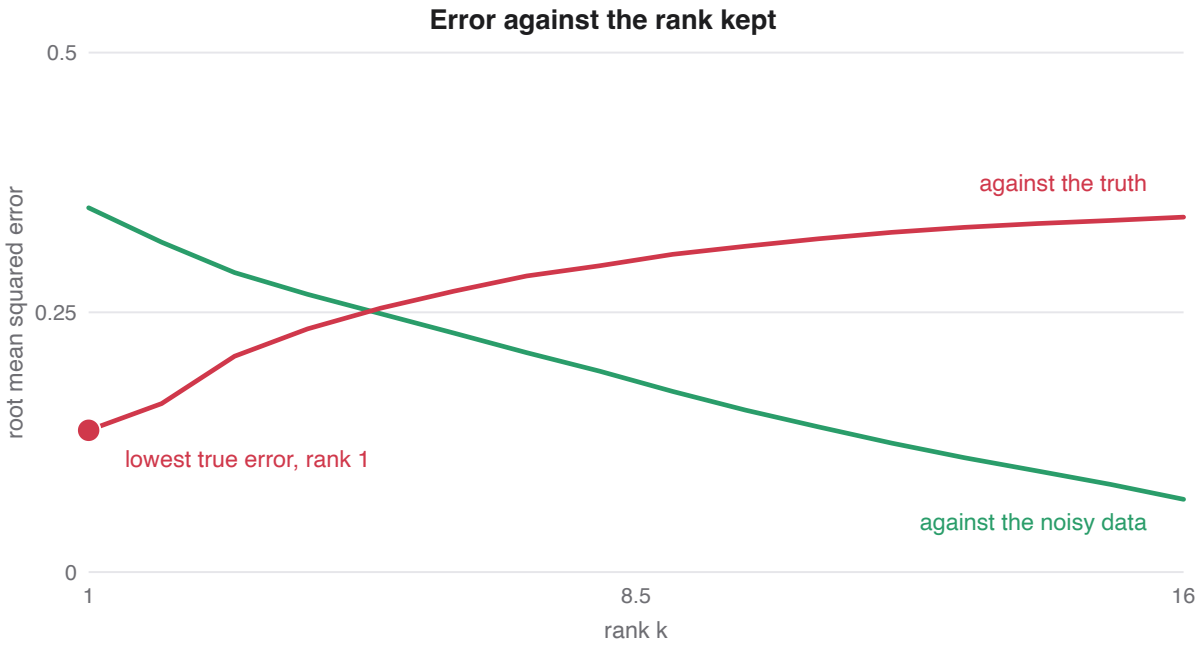
A6 Denoising

Suppose the data are structure plus noise: $Y = A + E$, where A has low rank. The large singular values mostly carry A, and the small ones mostly carry noise. Truncating at the right k removes more noise than signal.

Error against the true A makes a U-shape in k:

- too few layers discard real structure (bias);
- too many layers keep the noise (variance).

This is the bias-variance trade-off again, with k as the capacity knob. Note that the error against the noisy Y always falls with k, so it cannot choose k. As usual you need held-out entries or a noise estimate.



Truncating a noisy matrix. The green curve, error against the data you have, only falls. The red curve, error against the truth, has a minimum: past it you are reconstructing noise.

A7 The pseudoinverse and ridge through the SVD

With $X = U\Sigma V^\top$, least squares has the solution

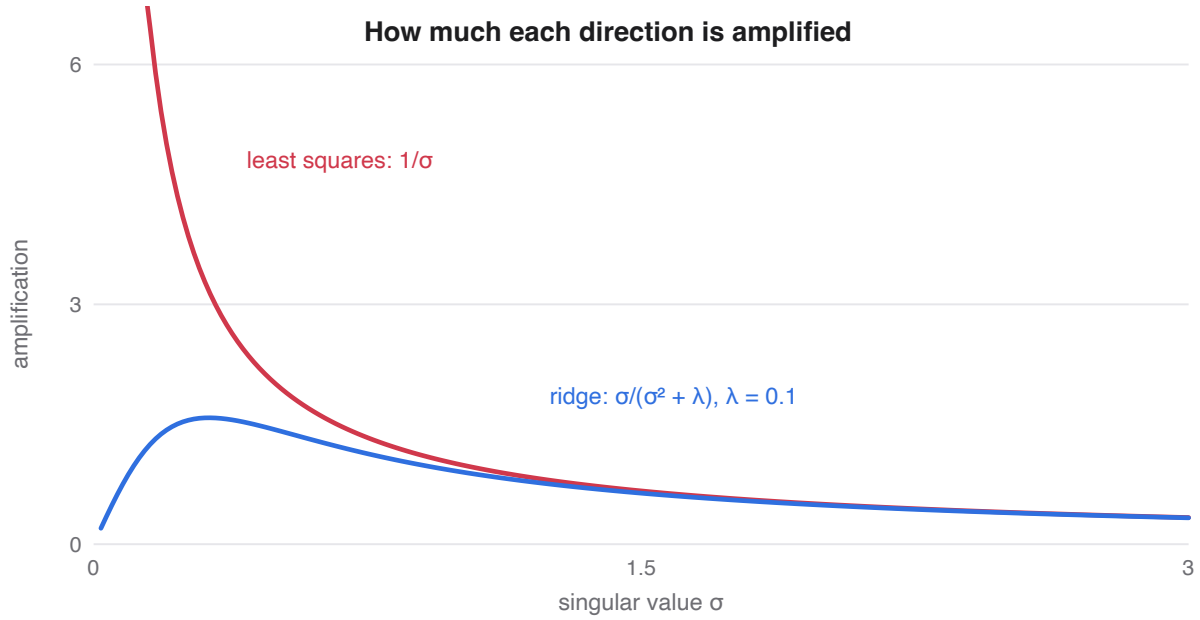
$$\hat{\beta} = X^+ y = V\Sigma^+ U^\top y, \qquad \Sigma^+ = \mathrm{diag}(1/\sigma_i) \ \text{ for } \sigma_i > 0.$$

When there are many solutions, this one has the **smallest norm**. That is the rule behind the second descent in double descent.

Each direction is amplified by $1/\sigma_i$, which is the numerical danger of tiny singular values. Ridge replaces that factor:

$$\frac{1}{\sigma_i} \longrightarrow \frac{\sigma_i}{\sigma_i^2 + \lambda},$$

which stays bounded as $\sigma_i \to 0$.



Why tiny singular values are dangerous, and how ridge fixes it. Least squares amplifies a direction by $1/\sigma$, which blows up. Ridge caps the amplification, and silences the weakest directions.

A8 Representation learning and embeddings

A **representation** is a new description of each example that makes later tasks easier. An **embedding** is a representation as a short vector, in which similar items lie close together.

PCA is the simplest case: replace $x \in \mathbb{R}^d$ by its k scores. Matrix factorization extends this to user-item tables: $R \approx UV^\top$ gives every user and every item a vector, and a predicted rating is their dot product.

Similarity in an embedding is usually measured by **cosine similarity**, which ignores length:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}.$$

A9 Limits of linear representations

- **Variance is not importance.** The top direction may be a nuisance, like lighting in images.
- **Units matter.** Without standardizing, PCA chases large units.
- **Only linear structure.** Data on a curve need more linear components than their true dimension.
- **Fit on training data only.** Fitting PCA on everything before splitting is leakage.

Also worth knowing

Dimensionality reduction in practice. Replacing d features by k scores is used for compression, for plotting, and as a preprocessing step that removes weak, noisy directions. Fit it inside the training fold, and choose k by what the downstream task needs and not only by variance kept.

Practice

P1. A 100×20 matrix has singular values $10, 5, 1$ and then zeros. What is its rank, and what fraction of the squared norm does a rank-one approximation keep?

> Rank 3. Total: $100 + 25 + 1 = 126$. Kept: $100/126 \approx 79.4\%$.

P2. Find the singular values of $\begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$.

> $A^\top A = \mathrm{diag}(4, 9)$, so the singular values are 3 and 2. They are never negative, and the sign goes into U.

P3. Why can the error against the noisy matrix not be used to choose k?

> Because it only ever decreases: $\|Y - Y_k\|_F^2 = \sum_{i>k} \sigma_i^2$. It rewards fitting the noise.

P4. With singular value $\sigma = 0.01$ and $\lambda = 0.1$, compare the OLS and ridge amplification factors.

OLS: $1/0.01 = 100$. Ridge: $0.01/(0.0001 + 0.1) \approx 0.0999$.